

Anvitha Mannepalli & Evan Litzer
COM S / CprE 4590X ,Cloud Security
Term Project Report
May 13, 2026

Secure Cloud File Storage System

1. Problem Definition

Cloud storage platforms are widely used for storing and sharing files, but most of them rely on the cloud provider to manage encryption ,which means the provider technically has access to your data. This creates a real privacy risk, especially for sensitive files. The problem we are solving is: how do you store files in the cloud in a way where even the cloud provider cannot read them?

Our system addresses this by implementing client-side encryption, where files are encrypted in the user's browser before they ever reach AWS. The cloud only ever sees ciphertext. The goal is to build a working prototype that demonstrates confidentiality and controlled access in a cloud storage environment ,two core concepts from this course.

We assume users have a browser and internet connection, and that AWS services (S3, Cognito, DynamoDB) are available. The system is designed for individual users who may want to store and share files securely without trusting the cloud with their plaintext data.

2. Conceptual Design

The system is composed of four main components that work together to handle authentication, encryption, storage, and sharing.

Authentication ,AWS Cognito manages user accounts and login. When a user logs in, Cognito issues identity credentials that are used to authorize access to AWS resources through an Identity Pool tied to IAM roles. This ensures only verified users can interact with S3 or DynamoDB.

Client-Side Encryption ,Before a file is uploaded, the browser encrypts it using the Web Crypto API with AES-256-GCM. A unique encryption key is generated per file. The key is stored alongside the file metadata in DynamoDB, tied to the user's identity. This means AWS S3 only ever receives and stores the encrypted ciphertext ,plaintext never leaves the user's device.

Storage ,AWS S3 holds the encrypted file blobs. Each file is stored under a path scoped to the uploading user. S3 bucket policies restrict access so users can only interact with their own files through the authenticated IAM role.

Access Control and Sharing ,DynamoDB maintains two tables: FileMetadata (stores file name, S3 key, encryption key, and owner) and SharedFiles (stores sharing records mapping a file to an authorized recipient). When a user shares a file, a new record is written to SharedFiles with the recipient's identity and the decryption key. The recipient can then fetch both the ciphertext from S3 and the key from DynamoDB to decrypt locally.

The overall flow is: User logs in → uploads file (encrypted in browser) → ciphertext goes to S3, metadata + key goes to DynamoDB → to share, owner writes a SharedFiles record → recipient logs in, fetches ciphertext + key, decrypts in browser.

3. Implementation Description

The system is built entirely as a frontend application ,there is no separate backend server. All AWS interactions happen directly from the browser using the AWS SDK for JavaScript.

Development Platform and Tools:

- React + Vite ,frontend framework and build tool
- AWS SDK for JavaScript (v3) ,for Cognito, S3, and DynamoDB interactions
- Web Crypto API ,built-in browser API used for AES-256-GCM encryption and key generation
- AWS Cognito User Pool + Identity Pool ,user authentication and IAM credential issuance
- AWS S3 ,encrypted file storage
- AWS DynamoDB ,file metadata and sharing records

Major Data Structures:

- FileMetadata table (DynamoDB): { fileId, owner, fileName, s3Key, encryptionKey, iv }
- SharedFiles table (DynamoDB): { shareId, fileId, sharedWith, encryptionKey, iv }
- Users table (DynamoDB): { userId, email } ,used for looking up recipients by email when sharing

Key Implementation Details:

Encryption is handled using `window.crypto.subtle.generateKey()` to create an AES-256-GCM key, and `window.crypto.subtle.encrypt()` to encrypt the file buffer before upload. The IV (initialization vector) is randomly generated per file and stored alongside the key in DynamoDB. Decryption on download uses `window.crypto.subtle.decrypt()` with the stored key and IV.

S3 uploads use the `PutObjectCommand` with the encrypted `ArrayBuffer` as the body. Downloads use `GetObjectCommand`, which returns the ciphertext that is then decrypted in the browser. IAM roles scoped per Cognito identity ensure users can only access their own S3 prefix.

4. User Guide

Installation:

- Clone the repository from GitHub
- Run npm install to install dependencies
- Create a .env file in the project root with your AWS configuration values (Cognito User Pool ID, Identity Pool ID, S3 bucket name, AWS region, DynamoDB table names)
- Run npm run dev to start the development server at localhost:5173

Using the System:

- Open the app in your browser and sign up or log in with your email and password through the Cognito-backed auth flow
- To upload a file: click the upload button, select a file from your device ,the app encrypts it in the browser and uploads the ciphertext to S3 automatically
- Your uploaded files appear under 'Your Files' ,click a file to download and decrypt it
- To share a file: select a file, enter the recipient's email address, and click Share ,the system looks up the user in DynamoDB and creates a sharing record
- The recipient will see the shared file under 'Shared With Me' when they log in, and can download and decrypt it the same way

5. Self Evaluation

Our main design goals were confidentiality (plaintext never reaches AWS), access control (users can only access their own or explicitly shared files), and a working end-to-end user flow. All three goals were accomplished.

Confidentiality is achieved ,AES-256-GCM encryption happens in the browser before upload, so S3 only holds ciphertext. This was verified by checking S3 directly and confirming the stored data is not human-readable.

Access control works through the combination of IAM roles scoped per Cognito identity and DynamoDB permission checks at the application level. A user cannot retrieve another user's file without a SharedFiles record explicitly granting access.

The end-to-end flow ,sign up, upload, download, share, recipient downloads ,was tested and confirmed working during development and during the in-class demo.

One limitation is that key management is relatively simple: encryption keys are stored in DynamoDB in a way that assumes DynamoDB itself is trusted. A more robust system would use

AWS KMS for key wrapping, which was part of the original plan but was descope'd due to time constraints. This is a clear direction for future improvement.

6. References

[1] X. Tang and L. Jin, "Data Splitting based Double Layer Encryption for Secure Ciphertext Deduplication in Cloud Storage," IEEE International Conference on Cloud Computing (CLOUD), 2024. <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10643895>

[2] A. Hartono et al., "CRISP: Confidentiality, Rollback, and Integrity Storage Protection for Confidential Cloud-Native Computing," IEEE International Conference on Cloud Computing (CLOUD), 2024. <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10643904>

[3] Y. Miao et al., "Efficient and Secure Spatial Range Query over Large-scale Encrypted Data," IEEE International Conference on Distributed Computing Systems (ICDCS), 2023. <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10272553>

7. Group Contributions

Evan Litzer:

- Designed and built the full React + Vite frontend application
- Set up all AWS infrastructure: Cognito User Pool and Identity Pool, S3 bucket, DynamoDB tables (FileMetadata, SharedFiles, Users), and IAM roles
- Implemented the file upload, download, and sharing features end-to-end
- Integrated the Web Crypto API for client-side AES-256-GCM encryption and decryption
- Presented slides covering problem definition, conceptual design, and core system architecture during the in-class pitch

Anvitha Mannepalli:

- Cloned the repository, configured the .env environment file, and set up the local development environment
- Performed integration testing of the full user flow ,sign up, file upload, download confirmation, and CORS debugging (resolved port conflict from 5174 to 5173)
- Verified end-to-end functionality including upload of test files and confirmed correct appearance in the 'Your Files' view
- Designed and prepared the implementation, user guide, demo, and evaluation slides for the in-class pitch presentation

- Authored the complete written project report